

```
#!/usr/bin/env python3
```

```
"""
```

```
ans_test_runner.py - runs the ANS Forth test suite (ans_tests/) against a  
live forth6809 image, over either a MAME "-bitb" TCP socket or a real  
serial-connected MECB6809 with the Forth binary in EPROM.
```

Why this exists (vs. minicom): forth6809's outer interpreter (QUIT) is line-based - it reads one line via ACCEPT, interprets it, then prints "\r\n ok" (or "\r\n ERROR n" on a thrown exception) before reading the next line. That "ok"/"ERROR" reply is a natural, content-based ready signal. Instead of pasting a whole file at a fixed baud rate and hoping end-of-character delays are long enough (the approach currently being fought with minicom, and the subject of the ongoing interrupt/RTS handshaking investigation), this script sends exactly one line, then BLOCKS until it sees that line's own "ok"/"ERROR" reply before sending the next one. That makes the pacing self-adjusting to however fast the target actually is - MAME under the debugger, MAME free-running, or real hardware - with no delay constant to tune per platform.

Three things this script does, per the project's outstanding ANS-test task list:

1. Loads the [IF]/[ELSE]/[THEN] tool-extension words first (00a_tool_ext_conditionals.fs) - forth6809.asm doesn't implement them, but ttester.fs needs them immediately (its own HAS-FLOATING / HAS-FLOATING-STACK checks use [IF]/[ELSE]/[THEN]).
2. Loads the shared preamble: ttester.fs, then 00_test_prelude.fs.
3. Loads one or more per-section *.tests.fs files. Each of those files is now self-contained: 09_outer_interpreter.tests.fs carries its own copy of GT1/GT2 rather than depending on section 13 having run first, and every section file opens with "MARKER Mnn" and closes by invoking Mnn, which erases everything that file itself defined (confirmed against forth6809.asm's MARKERW/DOMARKER - it snapshots and restores DPHERE/CODEHERE/VARHERE/LATEST, i.e. dictionary, value-space and the search-chain all at once). That makes each section file independently runnable, in any order, any number of times, from a single shared harness+prelude load.

Long test-file lines are re-wrapped before sending: forth6809's TIB is 80 bytes (TIBBUFL EQU 80 in forth6809.asm), but several *.tests.fs lines run past 1000 characters (many "T{ ... }T" blocks concatenated on one source line, harmless to a file-based INCLUDE but fatal to a terminal that ACCEPTs 80 bytes per line). Re-wrapping only ever splits at whitespace outside of S"/."-delimited strings, so it never changes the

token stream Forth sees, and never splits a "\"" line comment (which would turn its tail into live code on the next line).

Usage examples:

```
# Against MAME (spawns it, using the same -rs232 null_modem -bitb
# socket approach as run_all_tests.sh):
python3 ans_test_runner.py --target mame \
    --sections 08 09 10 11 12 13 14 15 16 17 18 19 20 21 23 24 26

# Against real hardware over a serial port:
python3 ans_test_runner.py --target serial --serial-port /dev/ttyUSB0 \
    --baud 19200 --sections 15
```

Run with --help for every option and its default.

"""

```
import argparse
import re
import socket
import subprocess
import sys
import time
from pathlib import Path
```

```
# -----
# Constants matching forth6809.asm
# -----
TIBBUFL = 80          # forth6809.asm: TIBBUFL EQU 80
SAFE_LINE_WIDTH = 72  # leave headroom below TIBBUFL for CR and slop
```

```
OK_MARKER = b"  ok"
ERROR_MARKER = b"  ERROR "
```

```
# tttester.fs's own failure messages (from ERROR1's S" ... " text, per
# tttester.fs read directly out of this project's ans_tests/ directory)
# plus forth6809's own system-level abort message. Any of these
# appearing in a section's captured transcript means that section did
# NOT fully pass, regardless of whether "ok" also appeared.
```

```
FAILURE_PATTERNS = [
    b"INCORRECT RESULT:",
    b"WRONG NUMBER OF RESULTS:",
    b"INCORRECT CELL RESULT:",
    b"NUMBER OF CELL RESULTS BEFORE '->' DOES NOT MATCH",
    b"NUMBER OF CELL RESULTS BEFORE AND AFTER '->' DOES NOT MATCH:",
    b"NUMBER OF RESULTS AFTER '->' BELOW ...}T SPECIFICATION:",
    b"INCORRECT FP RESULT:",
```

```

        b"WRONG NUMBER OF FP RESULTS:",
        ERROR_MARKER,
    ]

DEFAULT_SECTIONS = [
    "08", "09", "10", "11", "12", "13", "14", "15", "16",
    "17", "18", "19", "20", "21", "23", "24", "26",
]

# dpANS/Forth-2012 chapter numbering: 18 = FLOATING-POINT, 19 = FLOATING-POINT
# EXTENSIONS. This system has HAS-FLOATING FALSE, so these two files exercise
# a word set that isn't implemented at all - excluded by --no-float.
FLOATING_SECTIONS = {"18", "19"}

# -----
# Line re-wrapping: split long lines only on whitespace outside
# S"/.-delimited strings, and never touch a "\"" comment line.
# -----
def is_line_comment(stripped):
    leading = stripped.lstrip(" \t")
    return leading.startswith("\"\"") and (len(leading) == 1 or leading[1] in " \t")

def tokenize_preserving_strings(stripped):
    """Split on whitespace, but keep an S"/./C"/ABORT"-introduced string
    literal as one atomic token so a caller never mistakes a character
    inside it (a stray ':' or ';' in test text, say) for a real word,
    and ALSO fuse a whole "T{ ... }T" assertion into one atomic token
    (second pass, below). Shared by rewrap_line (so a rewrap point
    never lands inside a string or a T{}T block) and colon_delta (so a
    ':'/';' inside either is never mistaken for a real colon-definition
    boundary).

    The T{}T fusing matters because forth6809's ACCEPT/INTERPRET treat
    every physically-sent line as a hard parse boundary - there's no
    carry-over into "the next line" the way a file INCLUDE's REFILL
    provides one. T{ and }T aren't special to the Forth parser itself
    (ordinary space-delimited words), but a defining word and the name
    it parses from the word AFTER it (CREATE CR1, : DOES1, VARIABLE V,
    ...) must never land on two different rewrapped lines, or the
    second line's name token gets interpreted as a bare, undefined
    word while the defining word on the first line gets no name at
    all. Every T{...}T in this test corpus is already written as one
    self-contained unit by convention, so treating each one as atomic
    is always safe and directly prevents that failure mode for every
    defining word used inside one."""

```

```

raw_tokens = []
i = 0
n = len(stripped)
while i < n:
    while i < n and stripped[i] in " \t":
        i += 1
    if i >= n:
        break
    start = i
    # Detect a string-introducing word: S" / ." / C" / ABORT"
    # followed by exactly one space then a '''-terminated string.
    word_end = i
    while word_end < n and stripped[word_end] not in " \t":
        word_end += 1
    word = stripped[i:word_end]
    if word in ('S"', '."', 'C"', 'ABORT"') and word_end < n and stripped[word_end] == ' ':
        close = stripped.find('\'', word_end + 1)
        if close != -1:
            raw_tokens.append(stripped[start:close + 1])
            i = close + 1
            continue
    raw_tokens.append(stripped[start:word_end])
    i = word_end

```

```

tokens = []
j = 0
m = len(raw_tokens)
while j < m:
    if raw_tokens[j] == "T{":
        span = [raw_tokens[j]]
        j += 1
        while j < m:
            span.append(raw_tokens[j])
            closed = raw_tokens[j] == "}T"
            j += 1
            if closed:
                break
        tokens.append(" ".join(span))
    else:
        tokens.append(raw_tokens[j])
        j += 1
return tokens

```

```

def colon_delta(line):
    """Net change in open-colon-definition depth this line causes,
    scanning tokens outside string literals via tokenize_preserving_strings

```

```

so a ':' or ';' inside test text is never mistaken for a real one.
':' opens one level, ';' closes one - matched as exact tokens, not
substrings, so words like ':' inside a larger token don't count."""
stripped = line.strip("\r\n").strip()
if stripped == "" or is_line_comment(stripped):
    return 0
delta = 0
for tok in tokenize_preserving_strings(stripped):
    if tok == ":":
        delta += 1
    elif tok == ";":
        delta -= 1
return delta

def rewrap_line(line, width=SAFE_LINE_WIDTH):
    stripped = line.strip("\r\n")
    if len(stripped) <= width:
        return [stripped] if stripped != "" or True else []

    if is_line_comment(stripped.lstrip(" \t")):
        # A "\"" line comment: never split (its tail would become live
        # code on a continuation line). Just send it long - the target
        # discards it as a comment before hitting any length limit
        # inside its own text-parsing (parses one BL-delimited "word"
        # per token normally, but the whole rest of the line is inside
        # a comment, so nothing is ever an over-length token here).
        return [stripped]

    tokens = tokenize_preserving_strings(stripped)

    lines = []
    cur = ""
    for tok in tokens:
        candidate = tok if cur == "" else cur + " " + tok
        if len(candidate) > width and cur != "":
            lines.append(cur)
            cur = tok
        else:
            cur = candidate
    if cur:
        lines.append(cur)
    return lines

def rewrap_source(text):
    out = []

```

```

    for line in text.split("\n"):
        out.extend(unwrap_line(line))
    return out

# -----
# Transports
# -----
class SocketTransport:
    """Binds and listens BEFORE the caller spawns MAME, exactly like
    mame_listener.py does - MAME's own "-bitb socket.HOST:PORT" tries to
    CONNECT as a client first, so it connects straight in with no device
    path to discover and no startup race."""

    def __init__(self, host, port, accept_timeout):
        self.srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.srv.bind((host, port))
        self.srv.listen(1)
        self.srv.settimeout(accept_timeout)
        self.conn = None

    def accept(self):
        self.conn, addr = self.srv.accept()
        self.conn.settimeout(0.2)
        return addr

    def write(self, data):
        self.conn.sendall(data)

    def read_some(self):
        try:
            data = self.conn.recv(4096)
        except socket.timeout:
            return b""
        if data == b"":
            raise ConnectionResetError("connection closed by peer")
        return data

    def close(self):
        try:
            if self.conn:
                self.conn.close()
        finally:
            self.srv.close()

```

```

class SerialTransport:
    """Real serial-connected MECB6809, Forth binary in EPROM. Requires
    pyserial (pip install pyserial). rtscts=True enables hardware flow
    control at the host-OS/driver level, independent of whatever
    forth6809's own RTS/CTS firmware logic is doing - it is the
    project's stated eventual target alongside MAME."""

    def __init__(self, port, baud, rtscts):
        import serial # local import: only required for this transport
        self.ser = serial.Serial(
            port=port, baudrate=baud, timeout=0.2, rtscts=rtscts,
        )

    def write(self, data):
        self.ser.write(data)
        self.ser.flush()

    def read_some(self):
        return self.ser.read(4096)

    def close(self):
        self.ser.close()

# -----
# MAME process management (mirrors run_all_tests.sh's approach)
# -----
def launch_mame(args):
    cmd = [
        args.mame_bin, args.mame_system,
        "-rs232", "null_modem",
        "-bitb", f"socket.{args.listener_host}:{args.listener_port}",
        "-debug", "-debugscript", args.retrigger_cmd,
        "-window", "-resolution", "640x480",
    ]
    print(f"[mame] launching: {' '.join(cmd)}", file=sys.stderr)
    return subprocess.Popen(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

def assemble_rom(args):
    cmd = [
        args.lwasm_bin, "--6809", "--format=raw",
        f"--output={args.rom_out}", f"--list={args.rom_out}.lst",
        f"--define=UNITTESTS={args.unittests}",
        f"--define=SERIALPOLL={args.serialpoll}",
        args.asm_source,
    ]

```

```

    print(f"[asm] {' '.join(cmd)}", file=sys.stderr)
    subprocess.run(cmd, check=True)
    Path(args.rom_out).replace(args.rom_dest)

# -----
# Software (XON/XOFF) flow control - the counterpart to forth6809.asm's
# polling-build PUTXON/PUTXOFF. The target sends XOFF ($13) the instant
# a line is complete (ACCEPT's ADONE), before INTERPRET/compilation
# runs with KEY never polled at all, and XON ($11) once it's back in
# ACCEPT for the next line. An interrupt-driven-build target (real
# RTS/CTS hardware flow control) or real hardware never sends these
# bytes at all, so this is a harmless no-op there - flow.paused simply
# never becomes True.
# -----
XON_BYTE = 0x11
XOFF_BYTE = 0x13

class FlowControl:
    """Shared across the whole run (one instance threaded through every
    send_line_and_wait call) since it reflects the single target
    device's actual state, not anything scoped to one line or file.

    Starts paused=True, not False: nothing else tells this script when
    the target is actually ready to receive. Without this, the very
    first send races the target's own COLD/SIGNON-banner startup - the
    socket accepts long before the 6809 reaches ACCEPT's first PUTXON,
    and with no ring buffer in polling mode, bytes sent into that
    window just sit in (or overrun) the ACIA's single receive latch,
    corrupting whatever the target sees as its first line. Starting
    paused means the first wait_for_resume() call (at the top of the
    first send_line_and_wait) genuinely blocks - draining and logging
    the SIGNON banner - until the real startup XON arrives, or until
    wait_for_resume's own poll_timeout gives up and clears it (see
    there) for a target that never sends XON/XOFF at all (real
    hardware, interrupt-driven build)."""
    def __init__(self):
        self.paused = True

def scan_for_flow(chunk, flow):
    """Strip XON/XOFF bytes out of a just-read chunk, updating flow's
    paused state as a side effect. Callers must use the returned bytes
    for marker-matching/transcript logging, not the original chunk - a
    stray XOFF/XON byte left in could otherwise land inside what looks
    like an "ok"/"ERROR" marker or corrupt the logged transcript."""

```

```

    if XON_BYTE not in chunk and XOFF_BYTE not in chunk:
        return chunk
    out = bytearray()
    for b in chunk:
        if b == XOFF_BYTE:
            flow.paused = True
        elif b == XON_BYTE:
            flow.paused = False
        else:
            out.append(b)
    return bytes(out)

def wait_for_resume(transport, flow, log, poll_timeout=5.0):
    """Block until flow.paused is False, reading (and feeding through
    scan_for_flow) whatever the target sends meanwhile so a still-busy
    target - mid INTERPRET/compile, not polling KEY at all - never gets
    new bytes it has nowhere to put. poll_timeout is a safety net only,
    in case an XON is ever lost on the wire: it gives up waiting and
    lets the caller proceed rather than hang forever."""
    if not flow.paused:
        return
    deadline = time.monotonic() + poll_timeout
    while flow.paused and time.monotonic() < deadline:
        chunk = transport.read_some()
        if chunk:
            clean = scan_for_flow(chunk, flow)
            if clean:
                log.write(clean)
                log.flush()
            else:
                time.sleep(0.01)
    if flow.paused:
        # Gave up, not resumed: either the initial startup wait (see
        # FlowControl's docstring) found no XON because this target
        # never sends one, or a genuine XON was lost mid-run. Either
        # way, clear paused now rather than leaving it set - otherwise
        # every remaining byte of the current line would re-enter this
        # same poll_timeout wait one byte at a time, since this is
        # called from inside send_line_and_wait's per-character loop.
        flow.paused = False

# -----
# The line-at-a-time send/wait protocol
# -----
class LineResult:

```

```

def __init__(self, sent, echoed, ok, error, timed_out):
    self.sent = sent
    self.echoed = echoed
    self.ok = ok
    self.error = error
    self.timed_out = timed_out

def send_line_and_wait(transport, line, char_delay, reply_timeout, log, flow):
    """Send one line, terminated by CR (forth6809's ACCEPT/QUIT treat CR
    ($0D) as end-of-line and silently ignore LF - see ACCEPT's ALOOP:
    CMPB #13 -> ADONE, CMPB #10 -> BRA ALOOP - so CR alone is correct and
    sending CRLF would just cost one ignored byte per line for no
    benefit)."""
    payload = line.encode("ascii", errors="replace") + b"\r"

    wait_for_resume(transport, flow, log)
    if char_delay > 0:
        for b in payload:
            wait_for_resume(transport, flow, log)
            transport.write(bytes([b]))
            time.sleep(char_delay)
    else:
        transport.write(payload)

    buf = b""
    deadline = time.monotonic() + reply_timeout
    saw_ok = False
    saw_error = False
    while time.monotonic() < deadline:
        chunk = transport.read_some()
        if chunk:
            chunk = scan_for_flow(chunk, flow)
            if not chunk:
                continue
            buf += chunk
            log.write(chunk)
            log.flush()
            if OK_MARKER in buf:
                saw_ok = True
                break
            if ERROR_MARKER in buf:
                saw_error = True
                # keep draining briefly to catch the throw-code digits
                drain_deadline = time.monotonic() + 0.5
                while time.monotonic() < drain_deadline:
                    extra = transport.read_some()

```

```

        if extra:
            extra = scan_for_flow(extra, flow)
            if extra:
                buf += extra
                log.write(extra)
            drain_deadline = time.monotonic() + 0.5
        break
    timed_out = not (saw_ok or saw_error)
    return LineResult(payload, buf, saw_ok, saw_error, timed_out), buf

def send_file(transport, path, char_delay, reply_timeout, retries, log, transcrip
    flow, soft_timeout=False, grace_timeout=1.0):
    """Two distinct, known cases produce fewer "ok" replies than lines
    sent, and both are correct target behavior, not data loss or a
    transport bug - so neither should cost a full reply_timeout stall
    per line, and neither should be reported as a [FAIL]:

    1. Mid colon-definition (tracked here via colon_delta, purely from
       the source text - no target state needed). QOK only prints "ok"
       once STATE is back to 0, i.e. once the definition's ":" has been
       matched by a ";" - every interior line of a multi-line ": ... ;"
       gets no "ok" at all, in any file, not just the harness ones. A
       real compile-time error still prints immediately regardless of
       STATE, so a short grace_timeout (rather than 0) still catches
       that quickly without paying the full reply_timeout on every
       interior line of every multi-line definition.

    2. soft_timeout=True, for harness files with multi-line [IF]/[ELSE]
       blocks (ttester.fs's own HAS-FLOATING/HAS-FLOATING-STACK checks).
       [ELSE] (00a_tool_ext_conditionals.fs, the ANS standard's own
       reference implementation) skips a false branch by calling REFILL
       directly from inside its own BL WORD loop whenever it runs out
       of the current line - entirely inside the single INTERPRET call
       QLOOP is already waiting on. Every line [ELSE] skips past is read
       by REFILL, not a fresh QUERY/ACCEPT cycle, and QOK doesn't reply
       until the whole multi-line skip finally returns - one reply for
       the entire span, not one per physical line. This can't be
       predicted from the source alone (it depends on ENVIRONMENT?'s
       runtime answer), so it still costs a wait, but grace_timeout
       keeps that wait short instead of the full reply_timeout.

    In both cases attempt=1 is final - retrying is unsafe here regardless
    (see the mid-definition retry note in the module docstring history:
    resending would re-feed the same text into a still-open definition,
    or into REFILL's own skip, a second time)."""
    text = Path(path).read_text()
    lines = rewrap_source(text)

```

```

fail_hit = False
depth = 0
for line in lines:
    if line.strip() == "":
        continue
    depth = max(0, depth + colon_delta(line))
    expect_ok = (depth == 0)
    line_timeout = reply_timeout if expect_ok else min(reply_timeout, grace_t

attempt = 0
while True:
    attempt += 1
    result, buf = send_line_and_wait(transport, line, char_delay, line_ti
    transcript.extend(buf.split(b"\n"))

    if not expect_ok:
        if result.error:
            # A real compile-time error mid-definition - report
            # it, but don't retry (unsafe mid-definition) and
            # don't wait further; classify()'s transcript-wide
            # scan for " ERROR " still catches it either way.
            #
            # Critically: QUIT's CATCH/ABORT forces STATE back to
            # 0 (interpret) the instant any error fires, no matter
            # how deep colon_delta's static count thinks we are -
            # a definition that was aborted mid-body never reaches
            # its own ';', so without this resync the depth count
            # is left permanently offset. Every following line
            # would then be (wrongly) treated as "interior, no
            # reply expected" for the rest of the file: the
            # runner would stop waiting for real "ok" replies
            # entirely and fire the remaining lines at full speed
            # with only a grace_timeout pause each, flooding the
            # input buffer - this is what a full lockup after a
            # cascade of errors usually means, not a second bug.
            print(f"    [ERROR mid-definition] {line[:60]!r}", file=sys.s
            fail_hit = True
            depth = 0
            break

        if result.timed_out and soft_timeout:
            print(f"    [note] no immediate reply for: {line[:60]!r} "
                  f"(expected while inside a multi-line [IF]/[ELSE] skip)",
                  file=sys.stderr)
            break

        if result.timed_out and attempt <= retries:
            print(f"    [retry {attempt}] no ok/ERROR within {line_timeout}s

```

```

        file=sys.stderr)
        continue
    if result.timed_out:
        print(f"    [FAIL] gave up after {retries} retries on: {line[:60]}")
        fail_hit = True
    # A real reply came back (ok or ERROR) for a line we expected
    # one from - nothing to resync, depth is already trustworthy.
    break
return fail_hit

def classify(transcript_bytes):
    for pattern in FAILURE_PATTERNS:
        if pattern in transcript_bytes:
            return False
    return True

def run_section(transport, ans_dir, section, char_delay, reply_timeout, retries,
    matches = sorted(Path(ans_dir).glob(f"{section}_*.tests.fs"))
    if not matches:
        print(f"[skip] no test file found for section {section}", file=sys.stderr)
        return None
    path = matches[0]
    print(f"=== section {section} ({path.name}) ===")
    log_path = Path(log_dir) / f"{section}.raw.log"
    transcript = []
    with open(log_path, "wb") as log:
        timed_out_any = send_file(transport, path, char_delay, reply_timeout, ret
            flow, grace_timeout=grace_timeout)
    transcript_bytes = b"\n".join(transcript)
    passed = classify(transcript_bytes) and not timed_out_any
    print(f"    -> {'PASS' if passed else 'FAIL'} (log: {log_path})")
    return passed

def main():
    p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.Raw
    p.add_argument("--target", choices=["name", "serial"], required=True)
    p.add_argument("--ans-dir", default="ans_tests", help="directory containing a
    p.add_argument("--sections", nargs="+", default=DEFAULT_SECTIONS,
        help="section number prefixes to run, e.g. 08 09 10 (default:
    p.add_argument("--no-float", action="store_true",
        help=f"drop the floating-point section(s) ({', '.join(sorted(
            f"--sections before running - this system has no FLOATIN
            f"files exercise undefined words rather than being harml
    p.add_argument("--char-delay", type=float, default=0.0,
```

```

        help="extra seconds between individual characters within a li
p.add_argument("--reply-timeout", type=float, default=10.0,
        help="seconds to wait for 'ok'/'ERROR' after a line before tr
p.add_argument("--retries", type=int, default=2,
        help="retries per line before giving up (default: 2)")
p.add_argument("--grace-timeout", type=float, default=1.0,
        help="seconds to wait for a reply on a line that is known in
        "(interior lines of a multi-line colon-definition; defau
        "since it only needs to catch a genuine compile-time err
p.add_argument("--log-dir", default="ans_test_results")

# mame-target options (mirror run_all_tests.sh's flags/defaults)
p.add_argument("--asm-source", default="forth6809.asm")
p.add_argument("--lwasm-bin", default="lwasm")
p.add_argument("--mame-bin", default="./mecb6809")
p.add_argument("--mame-system", default="mecb6809")
p.add_argument("--rom-out", default="forth6809.bin")
p.add_argument("--rom-dest", default=str(Path.home() / "Library/Application S
p.add_argument("--retrigger-cmd", default="retrigger.cmd")
p.add_argument("--listener-host", default="127.0.0.1")
p.add_argument("--listener-port", type=int, default=2000)
p.add_argument("--serialpoll", type=int, default=1, choices=[0, 1],
        help="assemble with polling (1, default - avoids the interrup
        "under separate investigation) or interrupt-driven (0) s
p.add_argument("--unittests", type=int, default=0, choices=[0, 1],
        help="0 (default): a normal end-user image, which is what the
p.add_argument("--skip-assemble", action="store_true",
        help="reuse whatever ROM is already at --rom-dest instead of
p.add_argument("--mame-accept-timeout", type=float, default=30.0,
        help="seconds to wait for MAME's initial connection (default:

# serial-target options
p.add_argument("--serial-port")
p.add_argument("--baud", type=int, default=19200)
p.add_argument("--rtscts", action="store_true", help="enable hardware RTS/CTS

args = p.parse_args()

if args.no_float:
    dropped = [s for s in args.sections if s in FLOATING_SECTIONS]
    args.sections = [s for s in args.sections if s not in FLOATING_SECTIONS]
    if dropped:
        print(f"[--no-float] excluding section(s): {' '.join(dropped)}", file

Path(args.log_dir).mkdir(parents=True, exist_ok=True)

mame_proc = None

```

```

if args.target == "mame":
    if not args.skip_assemble:
        assemble_rom(args)
    transport = SocketTransport(args.listener_host, args.listener_port, args.
    mame_proc = launch_mame(args)
    try:
        addr = transport.accept()
        print(f"[mame] connected from {addr}", file=sys.stderr)
    except socket.timeout:
        print("FAIL: MAME never connected - check -rs232/-bitb support with -
        mame_proc.terminate()
        sys.exit(2)
else:
    if not args.serial_port:
        p.error("--serial-port is required for --target serial")
    transport = SerialTransport(args.serial_port, args.baud, args.rtscts)

flow = FlowControl() # shared for the whole run - see FlowControl's
                     # own docstring. A non-polling target (real
                     # hardware, or an interrupt-driven build with
                     # its own RTS/CTS) never sends XON/XOFF at all,
                     # so flow.paused simply never becomes True and
                     # this is a no-op there.

try:
    # Harness + preamble, loaded once, ahead of every section.
    harness_log = Path(args.log_dir) / "00_harness.raw.log"
    transcript = []
    with open(harness_log, "wb") as log:
        for fname in ("00a_tool_ext_conditionals.fs", "ttester.fs", "00_test_
            fpath = Path(args.ans_dir) / fname
            print(f"=== loading {fname} ===")
            send_file(transport, fpath, args.char_delay, args.reply_timeout,
                flow, soft_timeout=True, grace_timeout=args.grace_timeo

results = {}
for section in args.sections:
    results[section] = run_section(
        transport, args.ans_dir, section,
        args.char_delay, args.reply_timeout, args.retries, args.log_dir,
        flow, grace_timeout=args.grace_timeout,
    )

print("\n=== summary ===")
n_pass = sum(1 for v in results.values() if v)
n_fail = sum(1 for v in results.values() if v is False)
n_skip = sum(1 for v in results.values() if v is None)

```

```
    for section, v in results.items():
        label = "PASS" if v else ("SKIP" if v is None else "FAIL")
        print(f"    {section}: {label}")
    print(f"{n_pass} passed, {n_fail} failed, {n_skip} skipped")
    sys.exit(0 if n_fail == 0 else 1)
finally:
    transport.close()
    if mame_proc is not None:
        mame_proc.terminate()
        try:
            mame_proc.wait(timeout=5)
        except subprocess.TimeoutExpired:
            mame_proc.kill()

if __name__ == "__main__":
    main()
```